In previous chapters, we spent quite some time ensuring a high-security level for the Spring Boot applications in Polar Bookshop. However, Config Service was not one of them, and it's still unprotected. Even if it's a config server, it's still a Spring Boot application at its heart. As such, we can secure it using any of the strategies provided by Spring Security.

Config Service is accessed over HTTP by the other Spring Boot applications in the architecture. Before using it in production, we must ensure that only authenticated and authorized parties can retrieve configuration data. One option would be to use the OAuth2 Client credentials flow to secure the interactions between Config Service and applications based on an Access Token. It's an OAuth2 flow specific for protecting service-to-service interactions.

Assuming that applications will communicate over HTTPS, the *HTTP Basic* authentication strategy would be another viable option. When using this strategy, applications can be configured with the username and password via the properties exposed by Spring Cloud Config Client: `spring.cloud.config.username` and `spring.cloud.config.password`. For more information, refer to the official documentation for Spring Security (https://spring.io/projects/spring-security) and Spring Cloud Config (https://spring.io/projects/spring-cloud-config).

14.1.2 Refreshing configuration at runtime with Spring Cloud Bus

Imagine you have deployed your Spring Boot applications in a cloud environment like Kubernetes. During the startup phase, each application loaded its configuration from an external config server, but at some point